

Document number: P3661R0
Date: 2025-03-15
Audience: EWG, SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Attributes, annotations, labels

Attributes turned out to be unsuitable for the role of "small features", "small modifications to bigger features" or *annotations* from other languages. The need for a new kind of non-ignorable annotation pops up time and again in different places. Given that the addition of this new kind of entity seems inevitable, we recommend that only one syntax is used for all the cases that need it.

This currently concerns three features proposed for C++:

- annotations for reflection ([\[P3394R1\]](#)),
- the profiles framework ([\[P3589R0\]](#)),
- contract labels ([\[P3400R0\]](#)).

[\[P3394R1\]](#) has a model use case for a new kind of annotations: to be able to inject compile-time values to declarations so that they can be read by "metaprograms".

```
[[=timeout(10ms)]]  
void test_case("sorting_is_stable")  
{  
    // ...  
}
```

[\[P3589R0\]](#) proposes to use an attribute at a global namespace scope to indicate whether a given profile is enforced in the translation unit. This controls:

- whether some constructs cause a compiler error or not,
- whether some runtime checks are injected (this one is controversial).

There are good reasons to use an attribute for this purpose. But we also hear objections to this decision. One of them is about *ignorability* of attributes. Some implementations want to ignore all attributes (just ignore any content between `[` and `]`) while still providing the profiles framework.

Using a non-ignorable annotation would be a way out of this conflict:

```
[[=profiles::enforce(std::type)]];
```

It is a bit of a stretch, as there is no metaprogram that would be consuming the embedded data. But conceptually one could think of the profile processing as a metaprogram that consumes as data the names of the profiles.

Finally, contract assertions. Contract assertions themselves could have been attributes themselves. This was objected to, one objection being that we want the platforms that ignore attributes (whether this is legal or not) to still respect the contract assertions. But now [\[P3400R0\]](#) proposes to add *labels* as means of annotating contract assertions.

```
void f(int i)  
    pre <strict> (i != 0);
```

Here, `strict` is a compile-time value that controls what instrumentation code is injected. This again looks very close to annotations: some compile-time data is passed in to modify the property of a contract assertion:

```
void f(int i)
    pre [[=strict]] (i != 0);
```

Here again, the annotation syntax would reflect non-ignorability, and the "metaprogram consumes the data" interpretation wouldn't be too much of a stretch.

References

- [N2761] — Jens Maurer, Michael Wong, "Towards support for attributes in C++ (Revision 6)", (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2008/n2761.pdf>).
- [P2552R3] — Timur Doumler, "On the ignorability of standard attributes" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2552r3.pdf>).
- [P3394R1] — Wyatt Childers, Dan Katz, Barry Revzin, Daveed Vandevoorde, "Annotations for Reflection", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3394r1.html>).
- [P3400R0] — Joshua Berne, "Controlling Contract-Assertion Properties", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3400r0.pdf>).
- [P3589R0] — Gabriel Dos Reis, "C++ Profiles: The Framework", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3589r0.pdf>).